IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 10 April 2025, accepted 4 May 2025, date of publication 7 May 2025, date of current version 14 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3567762

RESEARCH ARTICLE

ForBac: A Static Analysis Approach With Forward and Backward Analysis for Precision in Floating-Point Computations

M. G. THUSHARA[1] AND K. SOMASUNDARAM[2]

[1]Department of Computer Science and Applications, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, Kollam 690525, India
[2]Department of Mathematics, Amrita School of Physical Sciences, Amrita Vishwa Vidyapeetham, Coimbatore 641112, India

Corresponding author: M. G. Thushara (thusharamg@am.amrita.edu)

ABSTRACT Accuracy in floating-point computations is critical for reliable software, especially in systems where precision impacts safety and performance. Inaccuracies from rounding errors and precision loss can lead to significant issues in applications like scientific computing and engineering simulations. To address these challenges, we introduce ForBac, a static analysis model designed to enhance floating-point precision by combining forward and backward analysis techniques. This dual approach improves the accuracy of floating-point computations compared to traditional methods. We evaluated ForBac on standard benchmarks, including Newton-Raphson and Jacobi Iteration methods, demonstrating its ability to maintain accuracy across complex computations. Results show that ForBac not only improves precision but also offers a scalable solution for analyzing floating-point operations in varied computational contexts.

INDEX TERMS Static analysis, floating-point, abstract interpretation, precision, transfer function.

I. INTRODUCTION

In numerous fields, from aerospace and automotive engineering to financial and scientific computing, numerical computations form the backbone of critical systems. Embedded software in these domains often relies on floating-point arithmetic to handle calculations that involve real numbers, enabling complex operations like simulations, control systems, and data analysis. However, due to the inherent limitations of floating-point representation—such as rounding errors and precision loss—the accuracy of these computations can be compromised, which poses risks, especially in safety-critical applications. Thus, ensuring reliable and precise floating-point calculations is paramount to the successful operation of these systems.

Over the years, researchers have sought methods to minimize errors [1], [2] in floating-point computations. Traditional approaches to mitigating these inaccuracies often rely on extensive testing and simulation, which are limited by the scenarios they can cover and the time-intensive nature of execution. More recently, static analysis techniques have gained attention as a proactive solution, as they allow potential numerical inaccuracies to be identified and rectified in code before it is deployed. Static analysis does not execute the program but instead analyzes its structure and behavior using mathematical models, which can be applied systematically to verify accuracy across a wide range of scenarios [3]. However, existing static analysis methods may fall short in fully capturing or correcting precision errors due to their reliance on a single direction of analysis, typically either forward or backward.

In response to this gap, this paper presents ForBac, a novel static analysis model designed to enhance the precision of floating-point computations. ForBac's unique approach combines both forward and backward analysis using Abstract Interpretation (AI) [4], enabling a more comprehensive detection and correction of errors. Forward analysis in ForBac tracks error propagation through each computation, starting from input variables and moving through intermediate results. This forward-tracing captures where and how inaccuracies arise in calculations. Complementarily, backward analysis adjusts these results to meet specified precision requirements,

The associate editor coordinating the review of this manuscript and approving it for publication was Mohamed Elhoseny.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

working backward from the desired accuracy at the output to the input variables. By using this dual approach, ForBac can not only detect precision issues but also refine values throughout computations to ensure they align with predefined accuracy standards [5], [6].

To demonstrate ForBac's effectiveness, we developed a prototype and tested it using standard benchmark algorithms, including the Newton-Raphson Method and the Jacobi Iteration Method. These algorithms are widely used in numerical analysis and serve as representative cases for evaluating precision in floating-point computations. Results from these benchmarks show that ForBac significantly improves precision across complex calculations and achieves accuracy levels that traditional methods may not consistently attain. Additionally, ForBac's integration with the IEEE-754 standard [7] ensures compatibility with widely used floating-point formats, making it applicable across various embedded and high-performance computing environments.

This paper is structured as follows: Section II provides background information on floating-point computation challenges and existing static analysis techniques. Section III describes the ForBac model in detail, including its forward and backward analysis framework. Section IV discusses the implementation of ForBac, detailing the tools and libraries used. Section V presents the validation and benchmarking process, highlighting ForBac's performance across different test cases. Section VI concludes with a summary of findings, insights into ForBac's contributions, and potential directions for future research.

By introducing ForBac, we aim to provide a reliable and scalable solution for enhancing floating-point accuracy, addressing a critical need in embedded systems and scientific computation.

II. BACKGROUND STUDY

Floating-point computations [8], [9] are essential in various domains such as engineering, finance, and scientific research. Despite their advantages, maintaining accuracy in floating-point arithmetic is challenging due to factors like precision loss [10] and rounding errors [11]. These challenges arise from the limitations of floating-point representation, which can lead to inaccuracies in applications where precision is crucial, especially in safety-critical fields.

To overcome floating-point inaccuracies, researchers have explored multiple strategies. Traditional solutions, such as rigorous testing and extensive simulations, often fall short in terms of coverage and computational efficiency. An alternative proactive approach is static analysis [12], which examines source code [13] without execution to identify and address potential precision issues. Unlike dynamic testing that focuses on specific cases, static analysis [14], [15], [16] provides a holistic evaluation of the program, allowing developers to address precision challenges systematically across various scenarios.

By using mathematical techniques such as interval arithmetic, static analysis facilitates the estimation of error bounds and optimization of floating-point computations [17], leading to the development of more dependable software for critical applications.

A. UNITS IN LAST PLACE (ULPS)

Floating-point errors are commonly measured using a metric known as **units in the last place (ulps)** [18]. An ulp represents the smallest difference between two consecutive floating-point numbers at a specific scale. It provides a standardized measure of how rounding errors affect numerical computations. For instance, in critical applications like aerospace or finance, where small errors can propagate into significant inaccuracies, quantifying errors in terms of ulps helps developers evaluate and mitigate precision loss effectively.

Static analysis tools [19], [20], [21], [22], [23], [24] such as GAPPA [25] and Daisy [26] utilize ulps to quantify precision loss during computations. This metric provides developers with a clear understanding of how rounding errors propagate through operations, helping to identify and resolve numerical inaccuracies. By tracking these errors, improvements can be made to ensure computations meet the required level of accuracy for their intended applications.

B. STATIC ANALYSIS TOOLS

Several static analysis tools [25], [26], [27], [28], [29], [30], [31], [32] have been introduced to enhance floating-point precision by identifying potential issues and providing optimization techniques. These tools focus on:

- **Error Detection**: Detecting rounding and precision loss (e.g., ASTRÉE [27]).
- **Precision Tuning**: Enhancing computational accuracy (e.g., Daisy [26]).
- **Error Propagation**: Assessing how errors propagate through calculations (e.g., GAPPA [25]).
- **Verification**: Ensuring compliance with accuracy requirements (e.g., ROSA [32]).

Table 1 summarizes these tools, highlighting their features, supported languages, and performance considerations.

C. FORWARD AND BACKWARD ANALYSIS

Static analysis applies **data flow analysis** to track how data propagates through a program using a Control Flow Graph (CFG). The two primary methods include:

- **Forward Analysis**: Tracks computations from inputs to outputs, monitoring precision loss at each step.
- **Backward Analysis**: Works from desired outputs to inputs, ensuring the required precision to achieve specific accuracy goals.

These techniques provide a comprehensive understanding of precision behavior, enabling developers to refine floating-point operations effectively in critical applications [10], [33]. A sample illustration of arithmetic addition operation is shown in Figures 1 and 2.

TABLE 1. Summary of static analysis tools for floating-point computations.

Tool	Purpose	Language Compatibility	Data Types Supported	Scope	Execution Time (s)
ASTRÉE	Error Detection	C	IEEE754	Whole Program	–
Daisy	Precision Tuning	Scala/C	IEEE754, Fixed	Program Portion	–
Fluctuat	Error Analysis	C, ADA	IEEE754	Whole Program	8.26
FPTaylor	Precision Tuning	FPCore	binary64, binary32	Single Computational Kernel	130.24
FPTuner	Precision Tuning	FPCore	IEEE754	Single Computational Kernel	–
GAPPA	Error Propagation	Gappa	IEEE754, Fixed	Whole Program	1.22
PRECiSA	Error Range Estimation	PVS	IEEE754	Program Portion	247.94
ROSA	Error-Bound Verification	Scala	IEEE754, Fixed	Program Portion	115.26
SALSA	Accuracy Optimization	C	IEEE754	Whole Program	–
TAFFO	Precision Tuning	LLVM-IR	IEEE754	Program Portion	–

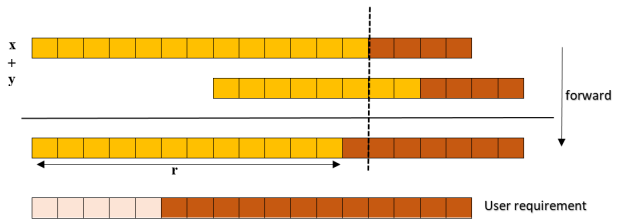

FIGURE 1. Illustration of forward analysis in floating-point addition.

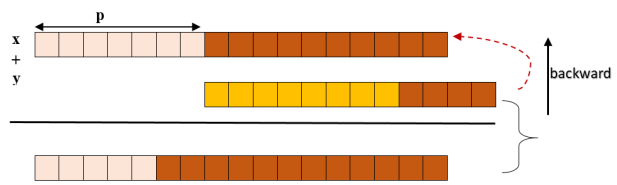

FIGURE 2. Illustration of backward analysis in floating-point addition.

Recent advancements in precision tuning and static analysis have led to significant improvements in computational efficiency and accuracy. Auto-tuning [34] strategies for mixed-precision computation, such as combining optimal code regions, have achieved speedups of up to 1.31×. The Best-5 block selection method balances execution time and error, emphasizing the need for careful variable selection to maintain accuracy. Additionally, a new single-precision Inexact Floating-Point Multiplier using Vedic algorithms [35] enhances computational precision and hardware efficiency, with FPGA-based designs addressing limitations in large-scale matrix multiplications. These innovations optimize performance while preserving accuracy in both software and hardware.

III. METHODOLOGY

To address precision challenges in floating-point computations, the ForBac model employs:

1) **Forward Analysis**: Tracks the propagation step-by-step, starting from input variables and moving through intermediate calculations.
2) **Backward Analysis**: Adjusts computations to ensure desired precision at the output, working back from output requirements to the input variables.

This dual approach ensures both detection and correction of precision errors across computational workflows.

A. PROTOTYPE MODEL

The ForBac prototype processes input code and calculates safe output ranges for numerical computations (refer Figure 3). Its key features include:

- Adherence to IEEE-754 floating-point standards.
- Transfer functions for basic operations (addition, multiplication, division, inverse, and power).
- Iterative precision refinement through combined forward and backward analysis.
- Use of interval domains to ensure conservative and safe over-approximations.

For instance, ForBac's transfer function for addition calculates precision based on the overlap of exponent ranges in the input intervals. This approach helps to predict how precision is affected at each computational step.

1) PROTOTYPE DESIGN

ForBac (refer Figure 3) analyzes code written in C with basic constructs like loops and arithmetic operations. It uses a parser to convert code into a structured format for analysis. It uses a custom parser to break down the code into structured representations for analysis. The key features of the ForBac model are as follows:

1) Embeds the **abstract semantics** (discussed in Section III-C) that adhere to IEEE-754 floating-point standards for data representation and manipulation. This allows to reason about the behavior of floating-point computation and its impact on precision.
2) Implement **transfer functions** (discussed in Section III-D) for fundamental arithmetic operations like addition, multiplication, division, inverse, and power functions. The precision of each operation is studied by the transfer function for predicting the accuracy of the final output.
3) Uses the combination of **forward and backward static analysis** (discussed in Section III-C). The iterative process between forward and backward analysis allows ForBac to track the changes in precision throughout the code, leading to a clearer accurate

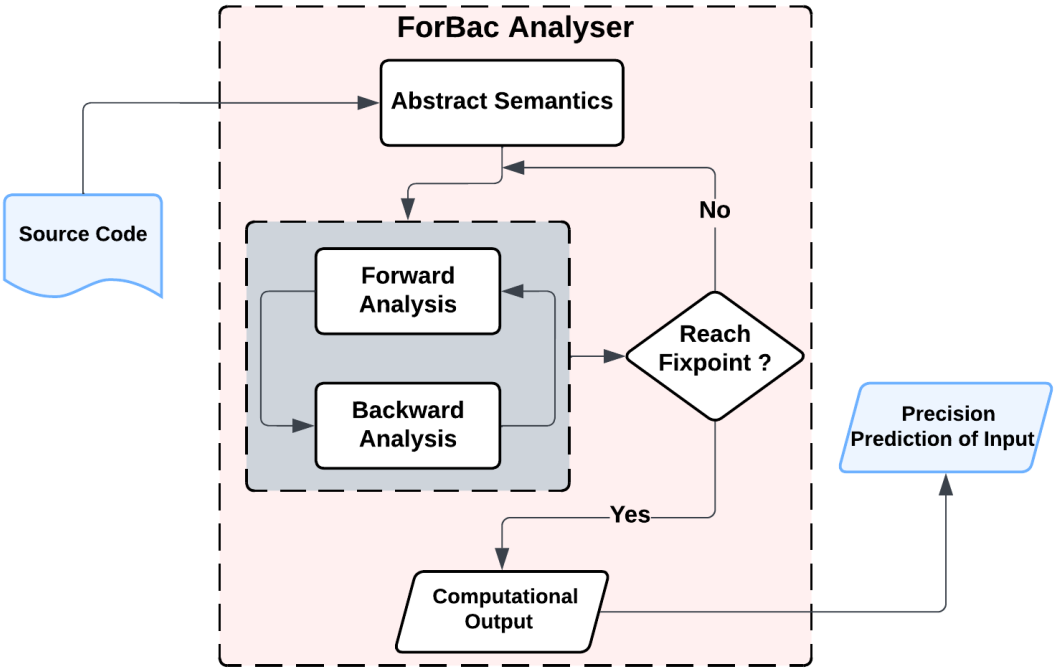

FIGURE 3. ForBac model architecture.

prediction of the minimum input precision needed to get a desired level of precision in the output.

4) Employs Interval domain within the conditional structures which provides an over-approximation of the results in all code branches of CFG, ensuring safer and conservative analysis.
5) Use of **join** at each loop entry, accelerates the convergence towards a fixed point during analysis. **Widening** is also used in case the analysis gets stuck in infinite loops.

The detailed grammar used by the parser is provided in A for further review.

B. ABSTRACT DOMAIN ANALYSIS

This section III-B discusses the details of the abstract domain employed by ForBac to perform static analysis of floating-point computations.

1) ABSTRACT DOMAIN FOR NUMERICAL ACCURACY

ForBac uses an interval-based abstract domain to represent the range of values possible for any numeric variable. The notation $[a, b]_p$ represents an abstract value, where:

- a and b are floating-point numbers that define a closed interval.
- p is an integer indicating the precision, which refers to the minimum number of digits guaranteed to be accurate within the given interval.

These intervals allow ForBac to analyze computations without requiring exact numerical values. Through a combination of forward and backward analyses, ForBac determines the least input precision required to ensure the desired accuracy of the output.

C. ABSTRACT SEMANTICS AND DOMAIN MODELING

This section III-C gives details on the core concept employed by ForBac for abstract domain analysis. Also, a discussion on Galois connection between the concrete domain of floating-point numbers and the abstract domain of intervals.

The floating-point numbers are represented with a sign bit, mantissa bits, and exponent bits. The precision of a representation is determined by the length of the mantissa. Define β_p as the set of all floating-point numbers with a mantissa precision length of p. Each element $x \in \beta_p$ can be represented as:

$$x = \pm b_0.b_1 b_2 \ldots b_p \times 2^e, \tag{1}$$

where b_0 denotes the sign bit, while $b_1, \ldots, b_p$ represent the bits of the mantissa, where $p \in \mathbb{N}$, and the exponent e lies within the range $[e_{min}, e_{max}]$ as defined in [36].

The IEEE-754 standard specifies widely used binary formats (e.g., single and double precision) and defines rounding methods [36]. Following the IEEE754 standard, we have 52+1 bits of mantissa, $e_{min} = -1022$ and $e_{max} = +1023$ in the binary 64 format. Also, defined

$$\beta = \bigcup_{p \in \mathbb{N}} \beta_p, \tag{2}$$

where β_p is a set of all floating-point numbers with precision p.

The rounding functions can be denoted as $\circ_{p,+\infty}$ (rounding up or ceiling), $\circ_{p,-\infty}$ (rounding down or floor), $\circ_{p,0}$ (rounding

towards zero), and $\circ_{p,\sim}$ (rounding to the nearest), each rounding numbers to precision p based on the specified mode. According to the IEEE 754 standard, the semantics of basic operations are defined by:

$$x \circledast_{p,r} y = \circ_{p,r}(x * y), \tag{3}$$

Here, $\circledast_{p,r}$ represents floating-point operations (such as $+$, $-$, $\times$, $\div$) performed using rounding mode r and precision p, while $*$ indicates the corresponding exact operation.

1) GALOIS CONNECTION - LINK BETWEEN ABSTARCT AND CONCRETE DOMAIN

Galois Connection [37], [38], [39] is a general framework for relating two ordered structures. ForBac focuses on the Galois Connection between partially ordered sets (POSETs).

A Galois Connection [40] is defined by two monotone functions - abstraction function (α) and concretization function (γ) as follows

$$\alpha : C \to A \tag{4}$$

$$\gamma : A \to C \tag{5}$$

α and γ are functions defined between partial orders $\leq$ on C and $\sqsubseteq$ on A where C is the concrete domain and A is the abstract domain, such that $\forall c \in C, a \in A$

$$\alpha(c) \sqsubseteq a \iff c \leq \gamma(a) \tag{6}$$

This equation (6) states that a concrete element c is related to an abstract element a through the Galois connection if and only if c is less than or equal to the concretization of a. The figures 4a and 4b can further illustrate the relationship between abstraction and concretization. They depict the monotonic nature of both functions and how the order is preserved through the connection.

a: GALOIS CONNECTION FOR INTERVAL ABSTRACTION

The abstract domain is an interval that defines a range of possible floating-point values for numeric variables. The minimum precision of the floating-point numbers in the interval is determined by the precision of an interval. The abstraction function maps a set of floating-point numbers to an interval which is the range of the minimum and maximum values, rounded based on a specified rounding mode (*discussed in defining the equation (3)*). Similarly, the concretization function maps an interval back to the set of all floating-point numbers within that range and across all possible precisions.

I_p denotes the collection of all intervals represented by floating-point numbers of a given precision p. Each element $i^\sharp \in I_p$ is denoted as $[f_1, f_2]_p$, an interval bounded by two floating-point values f_1 and f_2 and within the precision p. Then I is defined as the union of all such I_p sets across all possible precision levels. This represents the collection of all intervals for any floating-point precision. This defined as

$$I_p \ni [f_1, f_2]_p = \{f \in \beta_p : f_1 \leq f \leq f_2\} \text{ and } I = \bigcup_{p \in \mathbb{N}} I_p. \tag{7}$$

ForBac employs an abstract domain based on intervals to represent the possible range of values for numerical variables during program execution.

Definition 1: The abstract domain is defined as $D = \langle I, \sqsubseteq, \sqcup, \sqcap, \bot_I, \top_I \rangle$ represents all possible intervals used in ForBac, where I is the set of all intervals, $\sqsubseteq$ is the partial order relation [41] between intervals. The intervals are in the form $[a, b]_p$. The partial order $\sqsubseteq$ ensures that for any two intervals, one can be said to represent more precise values if it has both a smaller range and higher precision. This guarantees monotonic convergence of abstract computations. This is defined by

$$[a, b]_p \sqsubseteq [c, d]_q \iff [a, b] \subseteq [c, d] \text{ and } p \leq q \tag{8}$$

where p and q are the precision lengths of intervals $[a, b]$ and $[c, d]$, respectively.

In the context $[a, b]$ precedes$[c, d]$ if two conditions are met:

- The range of $[a, b]$ is entirely within the range of $[c, d]$, i.e., $[a, b] \subseteq [c, d]$.
- The precision p of $[a, b]$ is less than or equal to the precision q of $[c, d]$.
  - Precision reflects the guaranteed number of accurate digits.
  - If $p \leq q$, the interval $[a, b]$ has fewer guaranteed accurate digits than $[c, d]$, making $[c, d]$ less precise.

This formalization establishes that the abstract domain D is a POSET under the relation $\sqsubseteq$ which is stated in the proposition 1.

Proposition 1: The abstract domain $(D, \sqsubseteq)$ is a POSET.

Join ($\sqcup$) computes the least upper bound (lub) of two intervals and represents the most precise interval that encompasses both original intervals. Meet ($\sqcap$) computes the greatest lower bound (glb) of two intervals and represents the most precise interval that is contained within both original intervals.

Definition 2: The join ($\sqcup$) and meet ($\sqcap$) operations are defined by

$$[a, b]_p \sqcup [c, d]_q = [\circ_{r,-\infty}(u), \circ_{r,+\infty}(v)]_r \quad \text{with } r = \min\{p, q\},\ [u, v] = [a, b] \cup [c, d] \tag{9}$$

and

$$[a, b]_p \sqcap [c, d]_q = [u, v]_r \text{ with } r = \max\{p, q\}, \quad [u, v] = [a, b] \cap [c, d]. \tag{10}$$

The abstract domain also incorporates the notions of least and greatest elements. The Bottom element $\bot_I$ represents the interval with the least information, containing all possible floating-point values with infinitely low precision. It is denoted as $\emptyset_{+\infty}$, i.e., empty set with infinite precision. The Top element $\top_I$ represents the interval with the most information, encompassing all possible floating-point values with zero precision and denoted as $[-\infty, +\infty]_0$.

Definition 3: The least element $\bot$ and the greatest element $\top$ as follows:

$$\bot_I = \emptyset_{+\infty} \text{ and } \top_I = [-\infty, +\infty]_0. \tag{11}$$

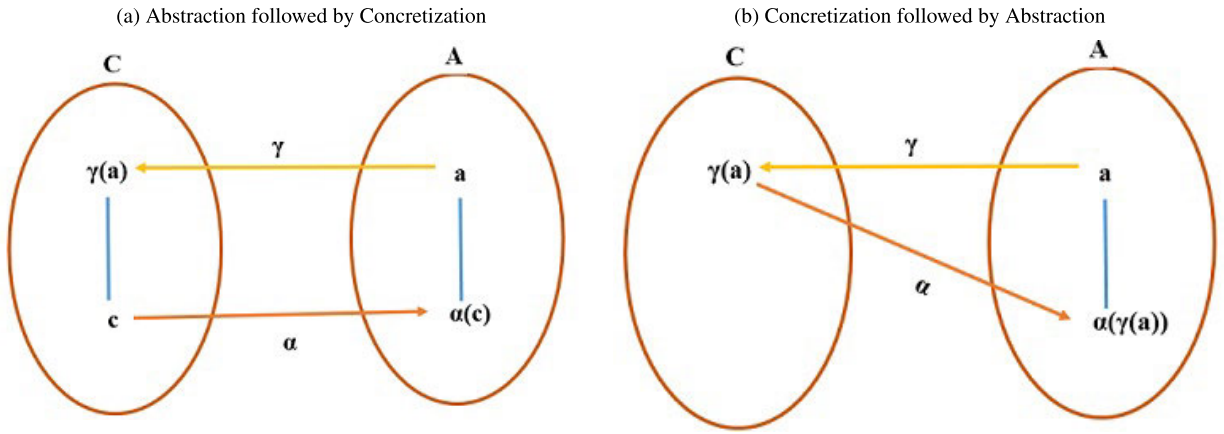

FIGURE 4. Demonstration of galois connection.

Since every element of D is ordered by $\sqsubseteq$ relation relative to the $\top$ and $\bot$, and both $\sqcup$ and $\sqcap$ operations are defined, the abstract domain D forms a complete lattice which is stated in the proposition 2.

Proposition 2: The POSET $(D, \sqsubseteq, \sqcup, \sqcap)$ is a bounded lattice.

The abstraction function α defined in equation (12) maps sets of concrete floating-point numbers (B) to abstract interval values (I) with potentially varying precisions within the abstract domain.

Definition 4: The abstraction function α takes a set of floating-point numbers B and associates it with a single value in a domain I. This function can handle different precision levels denoted by p_i where $1 \leq i \leq n$.

Additionally, for this set B, x_{min} and x_{max} represent the minimum and maximum values respectively, and p denotes the minimum precision observed in B. This p is defined by the equation:

$$\alpha(B) = [\circ_{p,-\infty}\big(\min(B)\big), \circ_{p,+\infty}\big(\max(B)\big)]_p \quad \text{where } p = \min\{q \;\; B \cap \beta_q \neq \emptyset\}. \tag{12}$$

Thus, the abstraction function α captures the range of values in B while considering the least precise representation across all elements. It applies rounding operations to ensure the resulting interval adheres to the limitations of the chosen precision level.

b: EXAMPLE TO SHOW ABSTRACTION

Consider a set of four floating-point numbers, each with different levels of precision:

$$B = 2.25_4, 2.5_5, 2.875_6, 2.890625_7, \tag{13}$$

$$\min(B) = 2.25, \quad \max(B) = 2.890625, \tag{14}$$

$$\alpha(B) = [2.25, 3.0]_4. \tag{15}$$

This interval represents the range of values within B, rounded to a precision of 4 bits. The smallest value, 2.25, is rounded down, while the largest value, 2.890625, is rounded up based on the 4-bit precision constraint.

The concretization function, denoted as γ, is defined in equation (16) and serves as the inverse of the abstraction function. It maps an abstract interval back to a collection of specific floating-point numbers.

Definition 5: Let $\gamma : I \rightarrow \wp(\beta)$ be the inverse of α and $i^\sharp = [a, b]_p$. Concretization function $\gamma(i^\sharp)$ is defined as

$$\gamma(i^\sharp) = \bigcup_{q \geq p} \{x \in \beta_q \; a \leq x \leq q\}. \tag{16}$$

c: EXAMPLE TO SHOW CONCRETIZATION

Let the abstract interval be $i^\sharp = [2.25, 2.891]_6$. The concretization function $\gamma(i^\sharp)$ would include the binary representations of all floating-point values between 2.25 and 2.891 (inclusive), encompassing various precision levels (e.g., 6 bits, 7 bits, etc.). The binary representation of $\gamma(i^\sharp)$, i.e. $\gamma([2.25, 2.890625]_6)$ is represented as follows:

$$\begin{array}{lcl} 2.25_6 & \rightarrow & 0\ 10000000\ 001000 \\ 2.5_6 & \rightarrow & 0\ 10000000\ 100000 \\ 2.75_6 & \rightarrow & 0\ 10000000\ 001100 \\ \vdots & & \end{array}$$

Proposition 3: The abstract function α and the concretization function γ form a Galois connection.

Proof: Let p_i and p_j be the two minimal precisions corresponding to B_i and B_j respectively. $\alpha(B_i) \leq \alpha(B_j)$ when ever $p_i \leq p_j$. Therefore the abstract function $\alpha : \wp(\beta) \rightarrow I$ as the abstraction function is monotonic.

Similarly, let q_i and q_j be the two precisions corresponding to the two intervals $i^\sharp$ and $i^{\sharp 1}$ respectively. It is easy to see that $\gamma(i^\sharp) \leq \gamma(i^{\sharp 1})$ when ever $q_i \leq q_j$.

Now, using the functions of the equations (12) and (16), we construct the Galois connection as

$$\langle B, \subseteq, \cup, \cap, \bot_B, \top_B \rangle \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} \langle I, \sqsubseteq, \sqcup, \sqcap, \bot_I, \top_I \rangle \tag{17}$$

■

The Galois connection established in Equation (17) exhibits Galois isomorphism [40]. This indicates that every

element in the concrete domain of function behaviors has a unique and precise representation in the abstract domain, and vice versa. This characteristic makes the connection in Equation (17) particularly powerful for reasoning about program behavior using the abstract domain without compromising the accuracy of the analysis.

2) OPERATORS FOR TERMINATION

One of the key challenges in interval analysis for program verification is ensuring the termination. A lattice structure represents the program's state during the analysis, and a program variable at each program point is mapped to an interval. In a program with q variables and p program points, the total number of states (lattice elements) can potentially reach pq. This exponential growth in the lattice size could hinder the termination of the analysis.

3) WIDENING OPERATOR

To address this challenge, we use widening operators. When the analysis appears to be caught in an iterative process that isn't converging towards a fixed point, a widening operator is applied. Widening operators deliberately increase the width (range) of the intervals associated with variables. This can help the analysis escape from an infinite loop of refinement and potentially reach a termination point. But it also costs to lose some degree of precision in the analysis. The condition for widening operation ($\triangledown$) is defined as:

$$\forall a, b \in \hat{D}.a \sqcup b \sqsubseteq a \triangledown b \tag{18}$$

For all increasing chains $d_0 \sqsubseteq d_1 \sqsubseteq \ldots$, then ascending chain $d_0^{\triangledown} \sqsubseteq d_1^{\triangledown} \sqsubseteq \ldots$ will stabilize when $d_0^{\triangledown} = d_0$ and $d_{i+1}^{\triangledown} = d_i^{\triangledown} \triangledown d_{i+1}$.

These properties guarantee that the widening operator will eventually terminate the analysis while providing a guaranteed over-approximation of the least fixed point (most precise interval achievable). This over-approximated result, referred to as a post-fixed-point, offers a balance between precision and efficiency in interval analysis.

4) NARROWING OPERATOR

To recover some of the lost precision during the widening operation, a narrowing operator ($\triangle$) can be used as an optional second pass after the widening operator has established a post-fixed point. The narrowing operation needs to follow the conditions listed below:

$$\forall x, y \in \hat{D}.(y \sqsubseteq x) \Rightarrow y \sqsubseteq (x \triangle y) \sqsubseteq x \tag{19}$$

For all decreasing chains $x_0 \sqsupseteq x_1 \sqsupseteq \ldots$, the sequence $y_0 = x_0, \ldots y_{i+1} = y_i \triangle x_{i+1}$ converges. This convergence signifies that the narrowing process will stabilize after a finite number of steps, leading to a refined interval that captures the actual value range more precisely.

The combination of widening and narrowing operators offers a practical approach to interval analysis. The widening operator ensures termination, while the optional narrowing step can potentially refine the analysis results, leading to a balance between efficiency and precision.

D. TRANSFER FUNCTIONS

Transfer functions help track how values change as a program runs. They show how a program's state at one point translates into its state after a specific step or operation.

Key functions include:

- **Addition**: Combines intervals while considering overlapping exponents to refine precision.
- **Multiplication and Division**: Predicts precision loss from bit-shifting during computations.
- **Power Function**: Accounts for compounded precision loss when raising values to an exponent.

NOTE:

ForBac ensures monotonicity and strictness for the transfer functions. The transfer function is considered monotone if it preserves the order relationship within the abstract domain. If a and b are abstract values and $a \sqsubseteq b$, then transfer function $T(a) \sqsubseteq T(b)$ also holds. This property ensures that the analysis maintains consistency as it progresses through the program. Similarly, a transfer function is considered strict if it maps the bottom element ($\perp$) of the abstract domain to itself. The bottom element typically represents the state with the least information, hence strictness ensures that executing an instruction on a state with no information will not give any information and results in the bottom element again.

1) TRANSFER FUNCTION FOR ADDITION

This section details the transfer function for addition within the interval analysis of ForBac (This in detail has been previously published by us in [5]). This function computes the abstract interval representing the possible range of values for the sum of two variables with known interval representations.

Let $x = [a, b]_{p_1}$ and $y = [c, d]_{p_2}$ be two intervals, where p_1 and p_2 are the precision levels associated with each intervals respectively. The key aspect of the transfer function is to calculate the precision p of the resulting interval $z = x + y$. The precision p is confirmed based on the different cases, considering the exponential difference ($d = |e_2 - e_1|$) between the intervals:

$$p = \begin{cases} min(p_1, p_2), & \text{if } e_1 = e_2, \\ min(p_1 - 1,\ p_2 + d - 1), & \text{if } e_1 > e_2, \\ min(p_1 + d - 1, p_2), & \text{if } e_1 < e_2. \end{cases} \tag{20}$$

Note: $a = s_1 \cdot m_1 \cdot 2^{e_1}$, $b = s_1' \cdot m_1' \cdot 2^{e_1'}$, $c = s_2 \cdot m_2 \cdot 2^{e_2}$ and $d = s_2' \cdot m_2' \cdot 2^{e'_2}$. Here, we take the minimum of $e_1 = min(e_1, e_1')$ and $e_2 = min(e_2, e_2')$ as the exponent value of the interval x and y.

a: JUSTIFICATION & CORRECTNESS

The addition operation on intervals is a monotone function within the lattice domain. This monotonicity, combined with the Knaster-Tarski theorem, guarantees that the sequence

of precision values computed during iterative addition operations will eventually reach a fixed point.

Theorem 1: The precision p attains a fixed point in its domain.

Proof: The precision p defined above is a function from $N \times N$ to N. Let us assume that $p_1, p_2 > 1$. If $e_1 = e_2$ then p takes the minimum of p_1 and p_2. The exponent difference d is a positive integer and hence minimum of $p_1 - 1$ and $p_2 + d - 1$ is also a positive integer and p attains the value when $e_1 > e_2$. If $e_1 < e_2$ then p attains minimum among $p_1 + d - 1$ and p_2. The addition operation is repeated iteratively then the value of p fixed between p_1 and p_2 on each iterations. Therefore in all the cases, p attains a fixed value. ■

2) TRANSFER FUNCTION FOR MULTIPLICATION

This section details the transfer function that computes the abstract interval representing the possible range of values for the product of two variables with known interval representations (This in detail has been published in [5]).

As discussed in section III-D1, we use the same intervals x and y. Here, the transfer function estimates the precision p' of the multiplication resulting interval $z = x * y$. The transfer function is defined as:

$$p' = \left\{ min(p_1, p_2) + 1, \text{for all } e_1 e_2 \text{cases.} \right. \tag{21}$$

The equation (21) shows increment of 1 which reflects the potential bit lost during internal carry operations or intermediate rounding, and does not suggest a gain in actual representational precision.

a: JUSTIFICATION & CORRECTNESS

Similar to addition, the multiplication operation on intervals is also a monotone function within the lattice domain. Due to the finiteness of the precision lattice and monotonicity of the function, convergence is guaranteed as per the Knaster-Tarski fixed point theorem.

The definition of p' inherently ensures its convergence. Since it always takes the minimum existing precision and adds one, the resulting value will always be less than or equal to the previous precisions encountered. This guarantees that the precision cannot increase indefinitely, leading to convergence within a finite number of iterations.

3) TRANSFER FUNCTION FOR DIVISION

This section details the transfer function that computes the abstract interval representing the possible range of values for the quotient [6] of two interval variables. Division poses a unique challenge in interval analysis compared to addition and multiplication. While those operations primarily introduce potential precision loss, division can also lead to undefined behavior (division by zero) within the abstract domain.

As discussed in section III-D1, we use the same intervals x and y. The division transfer function estimates the precision (p'') of the resulting interval $z = x/y$. The precision p'' is confirmed based on the different cases, considering the exponential difference ($d = |e_2 - e_1|$) between the intervals:

$$p'' = \begin{cases} max(p_1, p_2), & \text{if } e_1 = e_2, \\ min(p_1 - 1, p_2) + d, & \text{if } e_1 > e_2, \\ min(p_1, p_2 - 1) + d, & \text{if } e_1 < e_2. \end{cases} \tag{22}$$

a: JUSTIFICATION & CORRECTNESS

The division operation on intervals is also a monotone function within the lattice domain (excluding division by zero). This monotonicity, combined with the Knaster-Tarski theorem, guarantees that the sequence of precision values computed during iterative division operations will eventually reach a fixed point.

The equation (22) provides a conservative estimate of the resulting precision while maintaining monotonicity. The guaranteed convergence of precision facilitates efficient analysis by reaching a fixed point within a predictable time frame.

4) TRANSFER FUNCTION FOR INVERSE

This section details the transfer function that computes the abstract interval representing the possible range of values for the inverse of an interval variable.

We take the x_{p_1} interval discussed in Section III-D1. The transfer function is defined to estimate the resulting precision (p''') of the inverse interval x^{-1} as follows:

$$p''' = \begin{cases} p_1 - 1, & \text{if } p_1 > 1 \\ p_1, & \text{otherwise.} \end{cases} \tag{23}$$

The equation (23) indicates that for intervals with precision greater than one, the inverse operation reduces the precision by one bit. This reflects the potential loss of accuracy during the calculation of the inverse. However, for intervals with a precision of one (already the lowest possible), the precision remains unchanged.

a: JUSTIFICATION & CORRECTNESS

The transfer function for the inverse function preserves the monotonicity property within the lattice domain (excluding zero). This, along with the Knaster-Tarski theorem, guarantees that the sequence of precision values computed during the iterative application of the inverse function will eventually reach a fixed point.

5) TRANSFER FUNCTION FOR POWER

The transfer function computes the abstract interval representing the possible range of values for a variable raised to a certain power, considering the original interval representation and the exponent value.

We take the x interval discussed in Section III-D1. The power operation can significantly impact the precision of the resulting interval. To account for this, we define the transfer function to estimate the resulting precision (p'''') of the power

function x^n (where n is an integer exponent) as follows:

$$p'''' = \begin{cases} p_1 - \sum_{i=1}^{n} i, & \text{if } p_1/2 > n, \\ p_1 - 1, & \text{if } p_1/2 \leq n. \end{cases} \tag{24}$$

a: JUSTIFICATION & CORRECTNESS

The power function iteratively compounds the input interval, and with each operation, the potential bit loss adds up. Therefore, the precision p'''' is decremented based on the total number of multiplicative steps. The transfer function for the power function preserves the monotonicity property within the lattice domain. This, along with the Knaster-Tarski theorem, guarantees that the sequence of precision values computed during the iterative application of the power function will eventually reach a fixed point.

6) TRANSFER FUNCTIONS FOR BACKWARD ANALYSIS

The objective of backward analysis is determine the minimum accuracy required for program inputs to ensure a desired level of precision in the final result.

ForBac employs the same transfer functions for addition, multiplication, division, inverse, and power operations in both forward and backward analysis. However, the key distinction lies in the application order. In backward analysis, these functions are applied in the reverse direction, starting from the final result and working backward towards the program inputs.

By iteratively applying the transfer functions in reverse order, the backward analysis progressively estimates the minimum precision necessary for each program variable's input value. This estimation ensures that when these inputs are propagated forward through the program using the same transfer functions, the final result will achieve the desired precision level.

The backward analysis uses the established transfer functions to navigate the program structure backward, uncovering the minimum input accuracy requirements that guarantee a predetermined level of precision in the outcome. This approach provides valuable insights for optimizing resource allocation and ensuring efficient program analysis.

IV. VALIDATION AND BENCHMARKING

This section demonstrates how the proposed method improves the precision of numerical computations. By applying the approach to various examples, we evaluate its accuracy and effectiveness. Each example highlights a specific aspect of the method, such as its ability to detect errors or optimize input precision requirements.

A. LOOP WITH ADDITION AND MULTIPLICATION

To showcase how the method works, we analyzed a simple program (as shown in Listing 1) containing repeated addition and multiplication operations within a loop. The objective was to track how precision changes as the program executes

```
(0) a = 0.0; c = 0.25; t = 0;
(1) Declare x[t] = [0,1]_32 for all t;
  while (t < t_max)
  {
    (2)  a = a * a;
    (3)  a = a * c;
    (4)  a = a + x[t];
    (5)  t++;
  }
(6) y = 1.5 * a + 1.0;
(7) Declare y = ⊥_16;
(8) End
```

Listing 1. Example code for execution.

and identify the minimum input precision required to achieve accurate results.

The process overview is as follows:

1) The program initializes variables and iteratively updates their values using basic arithmetic operations.
2) Forward analysis calculates the range of values for each variable after every operation.
3) Backward analysis determines the required precision for the initial inputs to ensure the final result meets the desired accuracy.

In backward analysis, we begin with user-specified assertions that set benchmarks for the desired accuracy at specific points in the program. These assertions help us determine the minimum input precision needed for a particular output accuracy.

During execution, the value of a is updated by adding $x[t]$ to it. Initially, a is represented as an interval $[0.0, 0.0]_{53}$, which signifies its value within a 53-bit precision range. The value of $x[t]$ is in the interval $[0.0, 1.0]_{32}$, meaning it has 32-bit precision. When we combine these two intervals using the "join" operation, the resulting interval becomes $[0.0, 1.0]_{32}$. This reflects the updated range of a after the operation.

As the program progresses, the analysis uses a "join" operation at the loop head (control point (2)) to account for all potential execution paths. This ensures that the analysis captures every possible behavior of the code. Over time, the precision of the interval representing a improves, reflecting the accumulation of errors as the program iterates.

To ensure that the iterative analysis terminates after a finite number of steps, we use a technique called **widening** [4], [42], [43], [44]. At control point (2)''', we introduce widening, which deliberately over-approximates the interval's bounds. This allows the analysis to converge quickly to a fixed point, ensuring that no further iterations would provide additional information. After the widening operation, the interval for a becomes $[0, 2]_{32}$, reflecting the over-approximation.

At the end of the analysis, we evaluate y at control point (6) to determine the outcome of the program within the specified precision levels. The combination of joins and widening

$$
\begin{array}{ll}
(2) & a = [0.0, 0.0]_{53} \times [0.0, 0.0]_{53} = [0.0, 0.0]_{53} \\
(3) & a = [0.0, 0.0]_{53} \times [0.25, 0.25]_{53} = [0.0, 0.0]_{53} \\
(4) & a = [0.0, 0.0]_{53} + [0.0, 1.0]_{32} = [0.0, 1.0]_{32} \\
(5) & a = [0.0, 0.0]_{53} \cup [0.0, 1.0]_{32} = [0.0, 1.0]_{32} \\
 & \\
(2)' & a = [0.0, 1.0]_{32} \times [0.0, 1.0]_{32} = [0.0, 1.0]_{31} \\
(3)' & a = [0.0, 1.0]_{31} \times [0.25, 0.25]_{53} = [0.0, 0.25]_{31} \\
(4)' & a = [0.0, 0.25]_{31} + [0.0, 1.0]_{32} = [0.0, 1.25]_{32} \\
(5)' & a = [0.0, 1.0]_{32} \cup [0.0, 1.25]_{32} = [0.0, 1.25]_{32} \\
 & \\
(2)'' & a = [0.0, 1.25]_{32} \times [0.0, 1.25]_{32} = [0.0, 1.5625]_{31} \\
(3)'' & a = [0.0, 1.5625]_{31} \times [0.25, 0.25]_{53} = [0.0, 0.390625]_{31} \\
(4)'' & a = [0.0, 0.390625]_{31} + [0.0, 1.0]_{32} = [0.0, 1.390625]_{32} \\
(5)'' & a = [0.0, 1.25]_{32} \bigtriangledown [0.0, 1.390625]_{32} = [0.0, 2.0]_{32} \\
 & \\
(2)''' & a = [0.0, 2.0]_{32} \times [0.0, 2.0]_{32} = [0.0, 4.0]_{31} \\
(3)''' & a = [0.0, 4.0]_{31} \times [0.25, 0.25]_{53} = [0.0, 1.0]_{31} \\
(4)''' & a = [0.0, 1.0]_{31} + [0.0, 1.0]_{32} = [0.0, 2.0]_{32} \\
(5)''' & a = [0.0, 2.0]_{32} \\
 & \\
(6) & y = [1.5, 1.5]_{53} \times [0.0, 2.0]_{32} + [1.0, 1.0]_{53} \\
 & = [0.0, 3.0]_{32} + [1.0, 1.0]_{53} = [1.0, 4.0]_{33}
\end{array}
$$

Listing 2. Forward analysis on code from listing 1.

ensures that the analysis captures all possible behaviors of the program within a reasonable timeframe.

1) FORWARD ANALYSIS

Forward analysis works in the opposite direction, starting from the initial conditions and moving toward the final results. In the sample code snippet (Listing 2), we assume that the input variable a and the constant c have 53-bit precision, following the standard Binary64 format [18].

At control point (1), an assertion is made that all elements of the array $x[t]$ fall within the interval $[0.0, 1.0]$ with 32-bit precision. This might represent sensor data or computed values with limited accuracy.

During the first iteration, the value of a remains unchanged at $[0.0, 0.0]_53$, as this is its initial value. When a is squared at control point (2), its precision remains the same. At control point (3), the result is multiplied by the constant c.

2) BACKWARD ANALYSIS

This section shows the backward analysis process using the sample code (Listing 3). It showcase how backward analysis efficiently reveals the minimum input precision required for a desired output accuracy.

Backward analysis begins at control point (7), represented as (-7) to indicate the reverse direction. We start with the assertion $y = \perp$ (an unknown value with infinite precision) and the result from forward analysis, $y = [1.0, 4.0]_{33}$. By combining these two using the "join" operation, we get the assertion $y = [1.0, 4.0]_{33} \cup \perp_{\infty} == [1.0, 4.0]_{16}$, setting the target precision for y to 16 bits.

At control point (-6), the operation $y = 1.5a + 1$ is evaluated in reverse using the inverse transfer function.

$$
\begin{array}{ll}
(-7) & y = [1.0, 4.0]_{33} \cup \perp_{16} = [1.0, 4.0]_{16} \\
(-7) & c = [0.25, 0.25]_{53} \\
 & \\
(-6) & a = (([1.5, 1.5])^{-1}) \times ([1.0, 4.0]_{16} - [1.0, 1.0]_{53}) \\
 & \qquad = [0.0, 2.0]_{15} \\
 & \\
(-5) & a = [0.0, 2.0]_{32} \cap [0.0, 2.0]_{15} = [0.0, 2.0]_{15} \\
(-5) & c = [0.25, 0.25]_{53} \\
(-4) & a = [0.0, 2.0]_{15}\big(+ [0.0, 1.0]_{32})\big)^{-1} = [0.0, 1.0]_{14} \\
(-4) & x[t] = [0.0, 2.0]_{15}\big(+ [0.0, 1.0]_{14}\big)^{-1} = [0.0, 1.0]_{15} \\
(-3) & a = [0.0, 1.0]_{14}\big(\times [0.0, 0.25]_{53}\big)^{-1} = [0.0, 4.0]_{14} \\
(-3) & c = [0.0, 1.0]_{14}\big(\times [0.0, 4.0]_{14}\big)^{-1} = [0.0, 0.25]_{15} \\
(-2) & a = [0.0, 4.0]_{14}\big(\times [0.0, 2.0]_{32}\big)^{-1} = [0.0, 2.0]_{14} \\
(-2) & a = [0.0, 4.0]_{14}\big(\times [0.0, 2.0]_{14}\big)^{-1} = [0.0, 2.0]_{15} \\
 & \\
(-5)' & a = [0.0, 2.0]_{15} \cup [0.0, 2.0]_{15} = [0.0, 2.0]_{15} \\
(-5)' & a = [0.25, 0.25]_{53} \cup [0.25, 0.25]_{15} = [0.25, 0.25]_{15} \\
(-5)' & x[t] = [0.0, 1.0]_{32} \cup [0.0, 1.0]_{15} = [0.0, 1.0]_{15} \\
(-4)' & a = [0.0, 2.0]_{15}\big(+ [0.0, 1.0]_{15})\big)^{-1} = [0.0, 1.0]_{14} \\
(-4)' & x[t] = [0.0, 2.0]_{15}\big(+ [0.0, 1.0]_{14}\big)^{-1} = [0.0, 1.0]_{15} \\
(-3)' & a = [0.0, 1.0]_{14}\big(\times [0.0, 0.25]_{15}\big)^{-1} = [0.0, 4.0]_{14} \\
(-3)' & c = [0.0, 1.0]_{14}\big(\times [0.0, 4.0]_{15}\big)^{-1} = [0.0, 0.25]_{15} \\
(-2)' & a = [0.0, 4.0]_{14}\big(\times [0.0, 2.0]_{15}\big)^{-1} = [0.0, 2.0]_{14} \\
(-2)' & a = [0.0, 4.0]_{15}\big(\times [0.0, 2.0]_{15}\big)^{-1} = [0.0, 2.0]_{15} \\
 & \\
(-1) & a = [0.0, 2.0]_{15} \\
(-1) & c = [0.0, 0.25]_{15} \\
(-1) & x[t] = [0.0, 1.0]_{15}
\end{array}
$$

Listing 3. Backward analysis on code from listing 1.

This yields $a = 1.5^1 y1$, which results in the interval $a = [0.0, 2.0]_{15}$ for the variable a.

In subsequent control points (-4) and $(-4)'$, we apply the transfer function for addition, considering different precision levels for each case. Similarly, at control points (-3) and $(-3)'$, we use the transfer function for multiplication to propagate precision requirements backward through the loop.

After two iterations of backward analysis, we reach a fixed point, indicating that no further iterations would yield new information. The final outcome is that to achieve y with 16-bit precision, the input a needs at least 15-bit precision. Additionally, the constant c and the elements of the array $x[t]$ must also have at least 15-bit precision.

a: BENEFITS OF BACKWARD ANALYSIS

- Forward analysis often requires running multiple trials with different input precisions to determine the right output precision.
- Backward analysis, on the other hand, directly identifies the minimum input precision needed to achieve a desired output precision. This makes it a more efficient approach, as it focuses resources on the areas of the program where higher precision is crucial.

By using assertions and propagating precision requirements backward through the program, backward analysis provides a clear picture of the minimum input precision required for accurate output.

```
(1) float b=102.0, a=2.0_15;

(2) while(b>1.0)
    {
        (3) b=b/a;
        (4) printf("%f",b);
    }
```

Listing 4. Code snippet with division operation.

B. DIVISION OPERATION

To validate the handling of division, we analyzed a program (shown in Listing 4) where a value is repeatedly divided by another within a loop. This scenario often introduces challenges due to rounding errors.

The process overview is as follows:

1) The program starts with two variables: one representing the numerator and another as the divisor.
2) Each iteration computes the division and updates the numerator.
3) Widening techniques were applied to ensure the analysis converges within a finite number of steps.

The example code snippet in Listing 4 uses two variables, a and b, initialized to 102.0 and 2.0_{15}, respectively. These values are mapped to abstract domains as $[102.0, 102.0]_{32}$ and $[2.0, 2.0]_{15}$. The code includes a while loop that repeatedly divides b by a. Static forward analysis is applied to this code, as shown in Listing 5.

In ForBac, predefined abstractions automatically calculate the abstract semantics based on the program's source code. These semantics represent the potential value ranges of variables and can be further optimized either by ForBac itself or manually by the user.

At control point (3) in Listing 5, the analysis calculates the updated value of b' by dividing b by a using interval division. The division's transfer function is applied, producing a new interval for b': $[51.0, 51.0]_{32}$. At the start of each iteration, a join operation ($\cup$) combines the value of b from control point (1) with the updated value from control point (3). This ensures that the analysis captures all possible execution paths.

After control point (3), a widening operation is introduced to over-approximate the value of b. The widening ensures that the iterative process finishes after a finite number of steps. In this case, after a few iterations and the widening operation, b stabilizes to a constant value. This indicates that further iterations would not yield new information, signaling the termination of the loop. At this point, the final result of the forward analysis for b is determined, as shown in Listing 5.

This example (Listing 5) demonstrates how ForBac effectively handles division operations while accounting for precision loss. The use of join and widening operations ensures a comprehensive analysis and efficient termination of the iterative process.

```
(1) float b=[102.0,102.0]_32,
    a=[2.0,2.0]_15;

(3)     b'= [102.0,102.0]_32/ [2.0,2.0]_15
        =[51.0,51.0]_32

    b=b ∪ b'
    =[102.0,102.0}_32 ∪ [51.0,51.0]_32
    =[51.0,102.0]_32

(3)' b''=[51.0,102.0}_32/ [2.0,2.0]_15
        =[25.5,51.0]_15

    b=b' ∪ b''
    =[51.0,102.0}_32 ∪ [25.5,51.0]_15
    =[25.5,51.0]_32

(3)'' b'''=[25.5,51.0]_32/ [2.0,2.0]_15
        =[12.75,25.5.0]_15

    b=b'' ▽ b'''
    =[25.5,51.0]_32 ▽ [12.75,25.5]_15
    =[12.0,26.0]_32

(3)''' b''''=[12.0,26.0]_32/ [2.0,2.0]_15
        =[6.0,13.0]_15

    b=b''' ∪ b''''
    =[12.0,26.0]_32 ∪ [6.0,13.0]_15
    =[12.0,26.0]_32
```

Listing 5. Division operation analysis.

```
(0) a = [1.0, 1.0]_53

(1) x = a^2

(2) y = x^3

(3) Assert y = ⊥_16

(4) End
```

Listing 6. Code snippet with power operation.

1) POWER FUNCTION

We also analyzed a case (shown in Listing 6) involving a variable raised to a power. This operation often exacerbates precision loss, particularly with higher exponents.

The process overview is as follows:

1) The initial input was squared and then cubed, simulating repeated power operations.
2) Forward analysis tracked how precision decreased with each step.
3) Backward analysis was used to determine the minimum required precision for the initial input to meet the output requirements.

The code snippet in Listing 6 demonstrates the use of the power function. We assume a default precision of 53 bits, corresponding to the double-precision floating-point type.

```
(0)  a = [1.0, 1.0]_53

(1)  x = [1.0, 1.0]^2_53
     = [1.0, 1.0]_52

(2)  y = [1.0, 1.0]^3_52
     = [1.0, 1.0]_50
```

Listing 7. Forward analysis with power operation.

```
(0)  a = [1.0, 1.0]_53

(1)  x = [1.0, 1.0]^2_53
     = [1.0, 1.0]_52

(2)  y = [1.0, 1.0]^3_52
     = [1.0, 1.0]_50
```

Listing 8. Backward analysis with power operation.

The analysis begins at control point (0), where the variable a is initialized with the interval $[1.0, 1.0]_{53}$. This interval represents the possible values for a with 53-bit precision.

At control points (1) and (2), the square and cube of a are calculated, respectively. Listing 7 illustrates how the transfer function for the power operation is applied and how it affects the precision of the result. At control point (1), the square of a, or a^2, is computed as $[1.0, 1.0]_{52}$, which shows a loss of one bit of precision due to the squaring operation. Similarly, at control point (2), the cube of a, or a^3, is computed as $[1.0, 1.0]_{50}$, resulting in an additional two-bit loss of precision.

Listing 8 shows how backward analysis is applied to this code snippet. The aim of backward analysis is to determine the minimum input precision required to achieve a desired output precision at the final result. By iterating the transfer function for the power operations in reverse, backward analysis progressively estimates the necessary precision for the input. This information is then propagated backward through the program, starting from the final control point and moving toward the initial one.

C. ERROR ESTIMATION AND OPTIMIZATION

This section explores error estimation techniques [45], [46], [47] within ForBac for numerical programs. It emphasizes the importance of considering rounding errors inherent to floating-point computations.

1) UNITS IN LAST PLACE (ulps) FOR ERROR MEASUREMENT

Rounding errors are a defining characteristic of floating-point arithmetic. We measure these errors in units-in-last-place (ulps) [18]. This unit reflects the significance of the least significant bit in a floating-point representation. Using a floating-point format with parameters β and p, a computing difference using p digits, the relative error of the result can be as large as $\beta - 1$. Let z' be the floating-point number that approximates z, then $|d_0.d_1 \ldots d_{p-1} - (z/\beta^e)|\beta^{p-1}$ is equivalent to $|z' - z|/ulp(z')$. So we can formulate the measuring error as $|z' - z|/ulp(z)$. Here, we define the ufp by

$$0 \neq r \in \mathbb{R} \Rightarrow ufp(r) = 2^{log_2 \lfloor r \rfloor} \tag{25}$$

2) ERROR PROPAGATION

We analyze the error propagation during addition and multiplication operations involving two floating-point numbers, x with p-bit precision and y with q-bit precision. For addition as shown in equation (26), the total error (ϵ_r) is the sum of individual errors in $x(\epsilon_x)$ and $y(\epsilon_y)$ (shown in equation (27)). The resulting precision (pr) is estimated by considering the ulps of both the result and the total error. error.

$$r_{pr} = x_p + y_q \tag{26}$$

$$\epsilon_x \leq 2^{i-p} \quad \text{and} \quad \epsilon_y \leq 2^{j-q} \tag{27}$$

where $i = ufp(x)$ and $j = ufp(y)$.

We can find the error in the result as

$$\epsilon_r = \epsilon_x + \epsilon_y \tag{28}$$

The precision in the result can be obtained as

$$pr = ufp(r) - ufp(\epsilon_r) \tag{29}$$

Similarly, for multiplication shown in equation (30), the combined error (ϵ_r) shown in equation (31) involves a combination of terms: product of one operand's error with the other operand, and the product of both operands' errors. The resulting precision pr (in equation 32) is again calculated based on the ufps of the result and the combined error.

For multiplication we have:

$$r_{pr} = x_p \times y_q. \tag{30}$$

We calculate the error in the product result as

$$\epsilon_r = (\epsilon_x \times y) + (\epsilon_y \times x) + (\epsilon_x \, \epsilon_y) \tag{31}$$

The precision in the product result can be calculated as

$$pr = ufp(r) - ufp(\epsilon_r) \tag{32}$$

3) INPUT PRECISION ESTIMATION USING BACKWARD ANALYSIS

The framework employs backward analysis to determine the minimum input precision required for a desired output precision. This analysis leverages the relationship between operand and result ulps during addition and multiplication operations.

Theorem 2: Prove that

$$ufp(x) \leq \frac{4\,\epsilon_r - ufp(y)\,x}{2\,y\,ufp(y)} \tag{33}$$

Proof: We begin by observing that

$$2 \cdot ufp(x) \cdot y + ufp(x) \cdot ufp(y) \leq 4 \cdot \epsilon_r - ufp(y) \cdot x.$$

The error bounds and unit in the first place (ufp) are given as:

$$\frac{1}{2} \cdot ufp(x) \leq \epsilon_x \leq ufp(x) \tag{34}$$

$$\frac{1}{2} \cdot ufp(y) \leq \epsilon_y \leq ufp(y). \tag{35}$$

By combining Equations (34) and (35), we obtain:

$$\frac{1}{2} \cdot ufp(x) + \frac{1}{2} \cdot ufp(y) \leq \epsilon_r \leq ufp(x) + ufp(y). \tag{36}$$

Now, examining the left-hand side of Equation (36):

$$\frac{1}{2} \cdot ufp(x) \leq \epsilon_r - \frac{1}{2} \cdot ufp(y) \tag{37}$$

$$ufp(x) \leq 2 \cdot \epsilon_r - ufp(y) \tag{38}$$

Since we performed a backward analysis by utilizing both the result and one operand in forward analysis, we will now apply a backward approach for multiplication. Using the result (r_{pr}) and operand (y_q) in Equation (30), we aim to find x_p, assuming $y \geq 0$.

$$x_p = r_{pr} \div y_q. \tag{39}$$

Using the ufp error in Equation (39), we have:

$$\frac{1}{2} \cdot ufp(x) \cdot y \leq \epsilon_x \cdot y \leq ufp(x) \cdot y \tag{40}$$

$$\frac{1}{2} \cdot ufp(y) \cdot x \leq \epsilon_y \cdot x \leq ufp(y) \cdot x. \tag{41}$$

Combining Equations (40) and (41) yields:

$$\frac{1}{4} \cdot ufp(x) \cdot ufp(y) \leq \epsilon_x \cdot \epsilon_y \leq ufp(x) \cdot ufp(y) \tag{42}$$

$$\frac{1}{2} \cdot ufp(x) \cdot y + \frac{1}{2} \cdot ufp(y) \cdot x \leq \frac{1}{4} \cdot ufp(x) \cdot ufp(y) \leq \epsilon_r \tag{43}$$

$$2 \cdot ufp(x) \cdot y + ufp(x) \cdot ufp(y) \leq 4 \cdot \epsilon_r - ufp(y) \cdot x \tag{44}$$

$$ufp(x) \leq \frac{4 \cdot \epsilon_r - ufp(y) \cdot x}{2 \cdot y \cdot ufp(y)} \tag{45}$$

This completes the proof. ■

In the forward analysis (Listing 2), we trace the evolution of precision as arithmetic operations are applied iteratively within a loop, showing how precision is degraded over time. Conversely, the backward analysis (Listing 3) starts from a desired output precision and propagates constraints in reverse to determine the minimum required input precision. This dual perspective highlights the strength of the ForBac model in both tracking and correcting precision loss. Furthermore, the use of widening and narrowing operators ensures convergence of the analysis within a finite number of iterations, allowing ForBac to compute sound over-approximations efficiently.

```
a=[0.0,0.0];//default precision is taken as 53
c=[0.25,0.25];
t=0;x=[0.0,1.0]#32;
while(t<50)
{
    print a;
    a=a*a;
    a=a*c;
    a=a+x;
    t=t+1;
}
print a;
y=[1.5, 1.5]*a+[1.0,1.0];
print y;
y ASSERT 16;
```

Listing 9. Input code with loop.

V. CASE STUDIES AND RESULTS

This section evaluates the effectiveness of the proposed transfer functions in achieving precise static analysis of numerical programs. We demonstrate their application on various benchmarks and analyze the resulting precision obtained for the output variables.

More of benchmark results are given in VI.

A. DEMONSTRATION USING LOOP WITH BASIC ADDITION AND MULTIPLICATION OPERATIONS

We demonstrate the application of the transfer functions for fundamental arithmetic operations (addition, multiplication) using sample input code snippets given in Listing 9. The corresponding outputs from our prototype analysis tool are presented to illustrate the precision computation process. We further explore a code example involving a loop with a series of addition and multiplication operations. Here, the forward analysis is performed first, followed by an assertion on the desired output precision. Subsequently, backward analysis is employed to predict the minimum required precision for the input variable.

This section presents a detailed case study. The goal is to achieve a specific precision (e.g., 16 bits) for the output variable y in a code snippet. We begin the analysis by assuming an initial precision of 53 bits for the input variable a. By performing both forward and backward analysis on the code, the framework predicts that an input precision of 15 bits for a would be sufficient to guarantee the desired 16-bit precision for the output y. This highlights the framework's capability to optimize input precision requirements for achieving the target output accuracy.

```
Ans:
a [0.0,0.0]53
a [0.0,1.0]32
a [0.0,1.25]32
Join
a [0.0,2.0]32
a [0.0,2.0]32
a [0.0,2.0]32
```

```
x=[1.5,1.5]#53;
i=0;
while(i<4)
{
    f=x*x*x-x-[1.0,1.0]#53;
    df=[3.0,3.0]#53*x*x-[1.0,1.0]#53;
    h=f/df;
    y=x-h;
    x=y;
    i=i+1;
}
 print x;
```

Listing 10. Code for newton raphson method.

```
y [1.0,4.0]33
backward
a [0.0,2.0]15
a [0.0,2.0]15
```

This showcases the practical application of ForBac in optimizing resource allocation by identifying the minimum necessary precision for input values.

B. CASE STUDY: NEWTON-RAPHSON METHOD WITH PRECISION ANALYSIS

This subsection investigates the application of ForBac to analyze the precision behavior of the Newton-Raphson method, a root-finding algorithm for real-valued functions.

1) PROBLEM SET UP

We consider a third-degree polynomial function $f(x) = x^3 - x - 1$ as the target function for root-finding. The initial input value x_0 is set as an interval $[1.5, 1.5]_{53}$, representing a starting approximation with 53-bit precision as shown in Lisiting 10.

ForBac uses the transfer functions to track the precision of the computed root approximation throughout the iterative process of the Newton-Raphson method. After four iterations, the framework obtains an approximate root within the interval $[1.347826, 1.347826]_{48}$, indicating a reduction in precision to 48 bits.

```
Output:-
{Entry: name=f value=[0.875000,0.875000]50}
{Entry: name=df value=[5.750000,5.750000]53}
{Entry: name=h value=[0.152174,0.152174]50}
{Entry: name=x value=[1.347826,1.347826]48}
```

This case study demonstrates the ability of ForBac to analyze the impact of the Newton-Raphson method on variable precision. The observed precision loss after each iteration highlights the importance of considering precision behavior during numerical computations.

C. CASE STUDY: JACOBI ITERATION METHOD WITH PRECISION ANALYSIS

This subsection showcases the application of ForBac to analyze the precision behavior of the Jacobi iteration method, a popular iterative algorithm for solving systems of linear equations.

```
    x0=[0.0,0.0]#53;
    y0=[0.0,0.0]#53;
    z0=[0.0,0.0]#53;
    i=1;
    while(i<5)
    {
        x1=[17.0,17.0]-y0+[2.0,2.0]*z0;
        x1=x1/[20.0,20.0];
        y1=-[18.0,18.0]-[3.0,3.0]*x0+z0;
        y1=y1/[20.0,20.0];
        z1=-[25.0,25.0]-[2.0,2.0]*x0+[3.0,3.0]*y0;
        z1=z1/[20.0,20.0];
        x0=x1;
        y0=y1;
        z0=z1;
        i=i+1;
    }
    print x0;
    print y0;
    print z0;
```

Listing 11. Code for jacobi itertaion method.

1) PROBLEM SET UP

We consider a system of three linear equations as shown in Listing 11:

$$20x + y - 2z = 17$$
$$3x + 20y - z = -18$$
$$2x - 3y + 20z = 25$$

ForBac applies the transfer functions to track the precision of the computed approximate solutions (x, y, and z) throughout the iterative process of the Jacobi method.

After five iterations, the framework obtains approximate solutions within the following intervals:

```
Output:-
{Entry: name=x0 value=[0.850000,0.850000]28}
{Entry: name=y0 value=[-0.900000,-0.900000]27}
{Entry: name=z0 value=[-1.250000,-1.250000]28}
```

These results indicate a significant reduction in precision for all variables compared to the initial values (likely 53 bits for double-precision floats). This highlights the potential for precision loss during iterative computations.

D. CHALLENGES

While ForBac provides strong theoretical guarantees on precision analysis, its integration in real-world environments poses challenges. These include handling of unstructured control flows- for example difficulty with loops containing dynamic bounds or recursion, overly conservative estimates due to interval widening, and limited compatibility with floating-point nuances such as NaNs or denormalized numbers.

VI. CONCLUSION

This research explored the design of static analysis techniques for numerical programs, particularly those involving floating-

point computations. We addressed the challenge of ensuring program accuracy by proposing a novel method based on both forward and backward static analysis. The key features include:

- Representing variable ranges with an **interval domain** that tracks precision throughout execution.
- Defining **transfer functions** for key arithmetic operations like addition, multiplication, division, and power function, ensuring precision is managed effectively at every step.
- Using **forward analysis** to trace how errors propagate and **backward analysis** to calculate the minimum input precision needed for accurate results.

The effectiveness of our approach is validated through classical benchmarks like the Newton-Raphson and Jacobi iteration methods. Results showed that ForBac could predict precision loss, refine computations, and improve the reliability of floating-point operations.

FUTURE DIRECTIONS

Future work will explore the integration of ForBac with real-time static analysis tools in IDEs and compilers. This would enable on-the-fly precision tuning during development. Additionally, adapting ForBac for distributed systems, where precision constraints must be coordinated across heterogeneous nodes, presents an interesting research direction. Machine learning-driven adaptive tuning strategies also offer potential for extending ForBac to dynamically evolving environments. Potential improvements to ForBac include:

- **Real-time Precision Tuning**: Integrating dynamic analysis for adapting precision during execution.
- **Scalability**: Extending the model to handle distributed systems and parallel computations efficiently.
- **Broader Applications**: Applying ForBac to fields that require high precision, such as climate modeling, financial forecasting, and scientific simulations.

Overall, this research establishes a foundation for static analysis of numerical programs that explicitly considers precision as a critical factor. The proposed framework paves the way for more precise and resource-aware analysis of numerical computations, improving the reliability and efficiency of software development processes.

APPENDIX A
GRAMMAR OF THE PROTOTYPE

In the Appendix A, we show the grammar of the prototype and some sample input code for the proposed prototype model discussed in Section III-A.

```
program: stmt_list  { root = $1; }
;
stmt_list: stmt {$$ = processStlst($1, NULL);}
| stmt stmt_list {$$ = processStlst($1,$2);}
| LBRACE stmt_list RBRACE {$$ = $2; }
;
block: stmt { $$ = processStlst($1, NULL); }
| LBRACE stmt_list RBRACE   { $$ = $2; }
;

stmt: PRINT expr SC { $$ = processPrint ($2); }
| VARIABLE ASSIGN expr SC   { $$ = processAssign($1, $3); }
| WHILE LPAR bexpr RPAR stmt_list   { $$ = processWhile ($3,$5); }
| IF LPAR bexpr RPAR block  { $$ = processIfel  ('f',$3,$5,NULL);}
| IF LPAR bexpr RPAR block ELSE block   {$ =processIfel('e',$3,$5,$7);}
| error  { printf("SyntaxError %d\n", line); }
;

expr: doble { $$ = $1; }
| in    { $$ = $1; }
| VARIABLE   { $$ = id($1); }
| expr PLUS expr { $$ = opr('+', 2, $1, $3); }
| expr MINUS expr    { $$ = opr('-', 2, $1, $3); }
| expr TIMES expr    { $$ = opr('*', 2, $1, $3); }
| expr DIVIDE expr   { $$ = opr('/', 2, $1, $3); }
| expr MT expr   { $$ = opr('m', 2, $1, $3); }
| expr JN expr   { $$ = opr('j', 2, $1, $3); }
| LPAR expr RPAR { $$ = $2; }
;

bexpr: expr LT expr { $$ = opr('<', 2, $1, $3); }
| expr GT expr  { $$ = opr('>', 2, $1, $3); }
| expr GE expr  { $$ = opr('g', 2, $1, $3); }
| expr LE expr  { $$ = opr('l', 2, $1, $3); }
| expr NE expr  { $$ = opr('n', 2, $1, $3); }
| expr EQ expr  { $$ = opr('q', 2, $1, $3); }
;
doble: DOUBLE   { $$ = processDouble($1,$1,defaultPrec); }
| LBRACK DOUBLE COMMA DOUBLE RBRACK SHARP INT   {$$=processDouble($2,$4,$7);}
| LBRACK DOUBLE COMMA DOUBLE RBRACK {$$=processDouble($2,$4,defaultPrec);}
| DOUBLE SHARP INT  { $$ = processDouble($1,$1,$3); }
;
in: INT { $$ = processInteger($1,$1); }
| LBRACK INT COMMA INT RBRACK  { $$ = processInteger($2,$4); }
;
```

APPENDIX B
MORE BENCHMARKS

A. TAYLOR SERIES

We first take Taylor Series and applied the transfer functions. The Taylor series expansion for a function $f(x)$ around a point $x = a$ is given by:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \ldots$$

Now, consider the first-order differential equation $\frac{dy}{dx} = y^3 + 2$ with an initial value of $y(0) = 3$. We aim to approximate the value of y at $x = 0.2$ using a second-order Taylor polynomial:

$$y(x+h) = y(x) + y'(x)h + \frac{y''(x)}{2!}h^2.$$

Now, let us initialize all the variables required using intervals as $x = [0.0, 0.0]_{53}$ and $h = [0.2, 0.2]_{53} - [0.0, 0.0]_{53} = [0.2, 0.2]_{53}$.
Then we have $y''(x) = 3y^2\frac{dy}{dx} = 3y^2(y^3+2)$.

$$\begin{aligned} y(0.2) &= y(0) + y'(0) \times 0.2 + \frac{y''(0)}{2!} \times 0.2^2 \\ &= y(0) + y'(0) \times 0.2 + y''(0) \times 0.02. \end{aligned}$$

We can calculate $y'(0)$ and $y''(0)$ as shown in Figure 12 and then calculate $y(0.2)$ as shown in Figure 13. Applying the transfer functions for multiplication, addition and power we get a precision of 47 for the final computed value of $y(0.2)$.

B. EULERS METHOD

Sometimes for solving the differential equations we use Euler's method. The equation to calculate the Euler's method is as follows

$$y_n = y_{n-1} + hf(x_i, y_i),$$

$$y'(0) = [3.0, 3.0]^3_{53} + [2.0, 2.0]_{53} = [29.0, 29.0]_{50}$$

$$y''(0) = [3.0, 3.0]_{53} \times [3.0, 3.0]^2_{53}(y'(0))$$

$$y''(0) = [3.0, 3.0]^3_{53} \times [29.0, 29.0]_{50}$$

$$y''(0) = [27.0, 27.0]_{50} \times [29.0, 29.0]_{50} = [783.0, 783.0]_{49}.$$

Listing 12. Taylor series- calculation of $y'(0)$ and $y''(0)$.

$$y(0.2) = [3.0, 3.0]_{53} + [5.8, 5.8]_{49} + [15.66, 15.66]_{48}$$

$$y(0.2) = [24.46, 24.46]_{47}$$

Listing 13. Taylor series- calculation of $y(0.2)$.

$$y_1 = y_0 + hf(x_0, y_0) = 1 + 0.3f(0, 1) = 1 + 0.3(0^2) = 1$$

$$y_2 = y_1 + hf(x_1, y_1) = 1 + 0.3(0.3^2) = 1.027$$

$$y_3 = y_2 + hf(x_2, y_2) = 1.027 + 0.3(0.6^2) = 1.135$$

Listing 14. Eulers method- steps.

$$y_1 = [1.0, 1.0]_53 + [0.3, 0.3]_{53} \times [0.0, 0.0]^2_{53} = [1.0, 1.0]_{53}$$

$$y_2 = [1.0, 1.0]_{53} + [0.3, 0.3]_{53} \times [0.3, 0.3]^2_{53}$$
$$= [1.0, 1.0]_{53} + [0.3, 0.3]_{53} \times [0.09, 0.09]_{52}$$
$$= [1.027, 1.027]_{52}$$

$$y_3 = [1.027, 1.027]_{52} + [0.3, 0.3]_{53} \times [0.6, 0.6]^2_{53}$$
$$= [1.027, 1.027]_{52} + [0.3, 0.3]_{53} \times [0.36, 0.36]_{52}$$

$$= [1.027, 1.027]_{52} + [0.108, 0.108]_{52} = [1.135, 1.135]_{51}.$$

Listing 15. Using abstract domain.

where $y' = f(x_i, y_i)$ is the slope of the line at (x_i, y_i).

1) NORMAL METHOD

Apply Euler's method with a step size of $h = 0.3$ to approximate y at $x = 0.9$, where $y(x)$ satisfies the initial value problem $y' = x^2$ and $y(0) = 1$.

With a step size of 0.3, we proceed with values $x_0 = 0$, $x_1 = 0.3$, $x_2 = 0.6$, and end at $x_3 = 0.9$. Figure 14 illustrates each calculation step.

2) USING ABSTRACT DOMAIN

Since the step size is 0.3, we can start the value of $x_0 = 0$, $x_1 = 0.3$, $x_2 = 0.6$ and stop at $x_3 = 0.9$. The abstract interval domain is of the form $[a, b]_p$ and the initial variable can be set as $h = [0.3, 0.3]_{53}$, $x_0 = [0.0, 0.0]_{53}$. By default, we use a precision of 53.

$$[1.0, 2.0]_{21}x = [5, 10]_{25}$$
$$x = [1.0, 2.0]^{-1}_{21} \times [5, 10]_{25}$$
$$x = [1.0, 2.0]_{20} \times [5, 10]_{25}$$
$$x = [5.0, 20.0]_{20}.$$

Listing 16. Example 1: A single linear equation.

$$[1.0, 2.0]_{21}x + [2.0, 2.0]_{21}y = [5.0, 10.0]_{21} \quad (1)$$
$$[0.0, 3.0]_{21}x + [1.0, 4.0]_{21}y = [8.0, 12.0]_{21} \quad (2)$$

from (1),
$$x = [1.0, 2.0]^{-1}_{21}([5.0, 10.0]_{21} - [2.0, 2.0]_{21}y) \quad (3)$$

Substituting (3) in (2),
$$[0.0, 3.0]_{21}([1.0, 2.0]^{-1}_{21}([5.0, 10.0]_{21} - [2.0, 2.0]_{21}y))$$
$$+[1.0, 4.0]_{21}y$$
$$= [8.0, 12.0]_{21}$$

$$[0.0, 3.0]_{21}([0.5, 1.0]_{20}([5.0, 10.0]_{21} - [2.0, 2.0]_{21}y))$$
$$+[1.0, 4.0]_{21}y$$
$$= [8.0, 12.0]_{21}$$

$$[0.0, 15.0]_{20} - [0.0, 12.0]_{20}y + [1.0, 4.0]_{21}y = [8.0, 12.0]_{21}$$
$$[-11.0, 4.0]_{19}y = [8.0, 12.0]_{21} - [0.0, 15.0]_{20}$$
$$y = ([-11.0, 4.0]^{-1}_{19})[8.0, 12.0]_{21} - [0.0, 15.0]_{20}$$
$$y = [-0.09, 0.25]_{18} \times [-142.0, 12.0]_{19}$$
$$y = [-1704.0, 3.0]_{18} \quad (4)$$

Substituting (4) in (2),

$$[0.0, 3.0]_{21}x + [1.0, 4.0]_{21} \times [-1704.0, 3.0]_{18} = [8.0, 12.0]_{21}$$
$$[0.0, 3.0]_{21}x + [-6816.0, 12.0]_{18} = [8.0, 12.0]_{21}$$
$$[0.0, 3.0]_{21}x = [8.0, 12.0]_{21} - [-6816.0, 12.0]_{18}$$
$$[0.0, 3.0]_{21}x = [-4.0, 8638.0]_{17}$$
$$x = [0.0, 3.0]^{-1}_{21} \times [-4.0, 8638.0]_{17}$$
$$x = [-27352.0, 2256.54]_{17}.$$

Listing 17. Example 2: Solving system of linear equations.

The script in Figure 15 shows the working of Euler's method using the interval abstract domain that we defined in our abstract model. Also, we use the transfer functions defined in our domain for arithmetic operations like addition, multiplication and power function. The precision we get for the final result is 51.

C. SOLVING LINEAR EQUATIONS WITH INTERVAL COEFFICIENTS

In this sub section we show two examples for obtaining the precision while solving the linear equations with interval coefficients. We apply the transfer functions to get the precision of the results.

Let us first consider a single linear equation [See Fig.16].

The Listing.17 shows the applications of transfer function while solving a linear equation with intervals as coefficients.

Here, we begin with a precision of 21 for all the intervals. After solving we get the interval value for x with a precision of 17 and we get the interval value for y with a precision of 18. The purpose of this code is to show how the proposed transfer functions can be applied in solving system of linear equations. This idea can be further extended to system of non-linear equations.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their valuable suggestions.

REFERENCES

[1] *Wall Street Journal*, Vancouver Stock Exchange, Vancouver, BC, Canada, Nov. 29, 1983.

[2] *European Space Agency, Ariane 501 Inquiry Board Report*, European Space Agency, 1996.

[3] E. Goubault, "Static analyses of the precision of floating-point operations," in *Proc. 8th Int. Symp. Static Anal.* Cham, Switzerland: Springer, 2001, pp. 234–259.

[4] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM SIGACT-SIGPLAN Symp. Princ. Program. Lang. (POPL)*, 1977, pp. 238–252.

[5] S. Kanagasabapathi and M. G. Thushara, "Forward and backward static analysis for critical numerical accuracy in floating point programs," *Comput. Sci.*, vol. 21, no. 2, pp. 163–177, 2020.

[6] M. Thushara and K. Somasundaram, "Static analysis on floating-point programs dealing with division operations," *Int. J. Adv. Comput. Sci. Appl.*, vol. 10, no. 10, pp. 422–425, 2019.

[7] W. Kahan, "IEEE standard 754 for binary floating-point arithmetic," *Lect. Notes Status IEEE*, vol. 754, no. 1776, p. 11, 1996.

[8] A. Herrou, F. de Dinechin, S. Letz, Y. Orlarey, and A. Volkova, "Towards fixed-point formats determination for faust programs," 2024, *arXiv:2403.06527*.

[9] A. Appel and A. Kellison, "VCFloat2: Floating-point error analysis in COQ," in *Proc. 13th ACM SIGPLAN Int. Conf. Certified Programs Proofs*, Jan. 2024, pp. 14–29.

[10] G. Magnani, L. Denisov, D. Cattaneo, G. Agosta, and S. Cherubin, "Precision tuning the rust memory-safe programming language," Tech. Rep., 2024.

[11] P. Langlois, M. Martel, and L. Thévenoux, "Accuracy versus time: A case study with summation algorithms," in *Proc. 4th Int. Workshop Parallel Symbolic Comput.*, Jul. 2010, pp. 121–130.

[12] Z. Lin, A. Sun, X. Zhang, and Y. Lu, "MixPert: Optimizing mixed-precision floating-point emulation on GPU integer tensor cores," in *Proc. 25th ACM SIGPLAN/SIGBED Int. Conf. Lang., Compil., Tools Embedded Syst.*, Jun. 2024, pp. 34–45.

[13] R. Parvathy and M. Thushara, "AST-based and token-based neural networks for source code classification: A comparative performance analysis," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jun. 2024, pp. 1–7.

[14] J. M. Kannimoola, B. Jayaraman, and K. Achuthan, "Semantics of temporal constrained objects," *J. Log. Algebr. Methods Program.*, vol. 119, Feb. 2021, Art. no. 100632.

[15] V. G. Lekshmy and J. M. Kannimoola, "Formal verification of IoT protocol: In design-time and run-time perspective," in *Inventive Communication and Computational Technologies: Proceedings of ICICCT 2020*. Cham, Switzerland: Springer, 2021, pp. 873–884.

[16] J. M. Kannimoola, B. Jayaraman, and K. Achuthan, "Run-time analysis of temporal constrained objects," in *Proc. Declarative Program. Knowl. Manag., Conf. Declarative Program.*, Würzburg, Germany. Cham, Switzerland: Springer, Jan. 2018, pp. 20–36.

[17] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, "Design and implementation of a special-purpose static program analyzer for safety-critical real-time embedded software," in *The Essence of Computation*. Cham, Switzerland: Springer, 2002, pp. 85–108.

[18] J.-M. Müller, "On the definition of ULP(x)," INRIA, Tech. Rep. RR-5504, 2005.

[19] F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, "Frama-C: A software analysis perspective," *Formal Aspects Comput.*, vol. 27, no. 3, pp. 573–609, May 2015.

[20] N. Damouche and M. Martel, "Mixed precision tuning with salsa," in *Proc. 8th Int. Joint Conf. Pervasive Embedded Comput. Commun. Syst. (PECCS)*, vol. 1, 2018, pp. 47–56.

[21] J. Hao, Y. Xia, F. Li, H. Yang, Z. Yi, B. Zhou, and J. Pang, "AMPT: Automatic mixed-precision tuning based on accuracy gain," in *Proc. 30th Asia–Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2023, pp. 329–338.

[22] D. B. Khalifa and M. Martel, "An evaluation of POP performance for tuning numerical programs in floating-point arithmetic," in *Proc. 4th Int. Conf. Inf. Comput. Technol. (ICICT)*, Mar. 2021, pp. 69–78.

[23] G. Tagliavini, A. Marongiu, and L. Benini, "FlexFloat: A software library for transprecision computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 1, pp. 145–156, Jan. 2020.

[24] D. Zoni and A. Galimberti, "Cost-effective fixed-point hardware support for Risc-v embedded systems," *J. Syst. Archit.*, vol. 126, Jul. 2022, Art. no. 102476.

[25] F. de Dinechin, C. Q. Lauter, and G. Melquiond, "Assisted verification of elementary functions using gappa," in *Proc. ACM Symp. Appl. Comput.*, Apr. 2006, pp. 1318–1322.

[26] E. Darulova, A. Izycheva, F. Nasir, F. Ritter, H. Becker, and R. Bastian, "Daisy-framework for analysis and optimization of numerical programs (tool paper)," in *Proc. Int. Conf. Tools Algorithms Construct. Anal. Syst.* Cham, Switzerland: Springer, 2018, pp. 270–287.

[27] P. Cousot, J. E. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, *The ASTRÉE Analyzer*. Cham, Switzerland: Springer, 2005, p. 21.

[28] D. Delmas, E. Goubault, S. Putot, J. Souyris, K. Tekkal, and F. Védrine, "Towards an industrial use of fluctuat on safety-critical avionics software," in *Formal Methods for Industrial Critical Systems*. Cham, Switzerland: Springer, 2009, pp. 53–69.

[29] W.-F. Chiang, M. Baranowski, I. Briggs, A. Solovyev, G. Gopalakrishnan, and Z. Rakamarić, "Rigorous floating-point mixed-precision tuning," in *Proc. 44th ACM SIGPLAN Symp. Princ. Program. Lang.*, Jan. 2017, pp. 300–315.

[30] F. de Dinechin, C. Lauter, and G. Melquiond, "Certifying the floating-point implementation of an elementary function using gappa," *IEEE Trans. Comput.*, vol. 60, no. 2, pp. 242–253, Feb. 2011.

[31] M. Moscato, L. Titolo, A. Dutle, and C. A. Munoz, "Automatic estimation of verified floating-point round-off errors via static analysis," in *Proc. Int. Conf. Comput. Saf., Rel., Secur.* Cham, Switzerland: Springer, 2017, pp. 213–229.

[32] E. Darulova and V. Kuncak, "Sound compilation of reals," in *Proc. 41st ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, Jan. 2014, pp. 235–248.

[33] M. Alqaradaghi and T. Kozsik, "Comprehensive evaluation of static analysis tools for their performance in finding vulnerabilities in Java code," *IEEE Access*, vol. 12, pp. 55824–55842, 2024.

[34] X. Ren, M. Kawai, T. Hoshino, T. Katagiri, and T. Nagai, "Auto-tuning mixed-precision computation by specifying multiple regions," *Concurrency Comput., Pract. Exper.*, vol. 37, no. 2, Jan. 2025, Art. no. e8326.

[35] S. Gokulalakshmi, B. G. Viswa, N. S. Niranjan, and S. Saravanan, "FPGA implementation of single precision inexact floating-point multiplier using vedic algorithm," in *Challenges in Information, Communication and Computing Technology*. Boca Raton, FL, USA: CRC Press, 2025, pp. 632–636.

[36] *IEEE Standard for Floating-point Arithmetic*, IEEE Standard 754-2008, 2008, pp. 1–70.

[37] A. Burrieza, I. P. de Guzmán, and A. Yuste-Ginel, "A multi-modal logic for Galois connections," vol. 14, pp. 155–176, Jul. 2022.

[38] P. Cousot and R. Cousot, "A Galois connection calculus for abstract interpretation," in *Proc. 41st ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, Jan. 2014, pp. 3–4.
[39] E. K. Horváth, R. Pöschel, and S. Reichard, "Invariance groups of functions and related Galois connections," *Beiträge zur Algebra und Geometrie/Contrib. Algebra Geometry*, vol. 62, no. 2, pp. 495–512, Jun. 2021.
[40] M. Nowak, "A proof of Tarski's fixed point theorem by application of Galois connections," *Studia Logica*, vol. 103, no. 2, pp. 287–301, Apr. 2015.
[41] B. Blanchet. (2002). *Introduction To Abstract Interpretation*.
[42] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, "A static analyzer for large safety-critical software," in *Proc. ACM SIGPLAN Conf. Program. Lang. Design Implement.*, May 2003, pp. 196–207.
[43] P. Cousot and R. Cousot, "Abstract interpretation and application to logic programs," *J. Log. Program.*, vol. 13, nos. 2–3, pp. 103–179, Jul. 1992.
[44] P. Cousot and R. Cousot, "Abstract interpretation frameworks," *J. Log. Comput.*, vol. 2, no. 4, pp. 511–547, 1992.
[45] M. Frechtling and P. H. W. Leong, *An FPGA-based Floating Point Unit for Rounding Error Analysis*. Imperial College Press, 2015.
[46] C.-P. Jeannerod and S. M. Rump, "On relative errors of floating-point operations: Optimal bounds and applications," *Math. Comput.*, vol. 87, no. 310, pp. 803–819, Jul. 2017.
[47] L. Titolo, M. Moscato, M. A. Feliú, A. Dutle, and C. Muñoz, "Floating-point round-off error analysis of safety-critical avionics software," *Challenges Softw. Verification*, vol. 238, pp. 197–220, Jul. 2023.

M. G.THUSHARA received the Ph.D. degree in computer science and engineering from Amrita Vishwa Vidyapeetham, Coimbatore, India, in 2021.

She was invited to work at the DALI Laboratory, University of Perpignan, France, from 2012 to 2013, focusing on abstract interpretation models. Since 2013, she has been an Assistant Professor with the School of Computing, Amrita Vishwa Vidyapeetham. Her research interests include program analytics, static analysis, software engineering, automatic code generation and assessment models, and large language models. From 2010 to 2013, she was a recipient of the Erasmus Mundus Scholarship for research at the University of Munich, Germany.

K. SOMASUNDARAM received the Ph.D. degree in mathematics from Bharathiar University, Coimbatore, India, in 2005, and the Postdoctoral degree from the Department of Computer Science, Turku University, Turku, Finland. He is currently the Chairperson and a Professor with the Department of Mathematics, Amrita School of Physical Sciences, Amrita Vishwa Vidyapeetham, Coimbatore. His research interests include multilinear algebra, graph theory, and program analysis.